

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning

COURSE CODE: MLA0403

FACULTY: Dr.Arul Raj A.M

Marks: 25 | Duration: 30 minutes

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

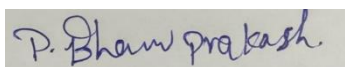
COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

Code of Conduct:

I P.Bhanu Prakash / 192425263 (Name/ Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Name of the student: P.Bhanu Prakash

Question 1: Error Analysis in Maximum Likelihood Estimation (MLE)

Scenario: A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

- Identify the conceptual error in the likelihood formulation.
- Explain how this mistake affects parameter estimation.
- Suggest the correct mathematical formulation and reasoning.
- How would using log-likelihood fix computational issues?

Detailed Engineering Solution & Analysis

Introduction: Maximum Likelihood Estimation (MLE) seeks parameter values θ that maximize the joint probability of observing the given dataset under an assumed probabilistic model. It relies fundamentally on probability theory and sample independence.

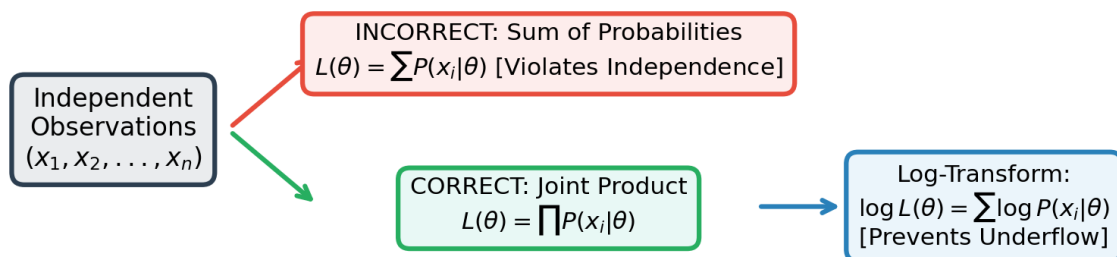
1. Conceptual Error: The fundamental mathematical mistake is using addition instead of multiplication for independent events. Under the assumption of independent and identically distributed (i.i.d.) observations, the joint probability density function $P(X|\theta)$ equals the product of individual probabilities: $P(x_1, x_2, \dots, x_n | \theta) = \prod P(x_i | \theta)$. Replacing multiplication with addition treats individual instances as mutually exclusive events rather than independent joint occurrences.

2. Impact on Parameter Estimation: Summing individual probabilities distorts the optimization landscape completely. The gradient of a sum depends only linearly on isolated terms, destroying parameter sensitivity and interactions across data points. It severely underestimates the penalty for points assigned low probabilities and yields poor, inconsistent parameter estimates regardless of sample size.

3. Correct Formulation: The correct likelihood function for a binary classifier (Bernoulli distribution) with predictions $y_i \in \{0, 1\}$ and model probability $p_i = P(y_i=1 | x_i; \theta)$ is given by:

$L(\theta) = \prod_{i=1}^N (p_i)^{y_i} * (1 - p_i)^{(1 - y_i)}$. This properly multiplies the individual conditional probabilities.

4. Advantages of Log-Likelihood: Taking the natural logarithm converts the product into a sum: $\log L(\theta) = \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$. This provides major computational advantages: (a) Numerical Stability: Prevents arithmetic underflow caused by multiplying thousands of small probabilities (< 1); (b) Simplified Gradient Optimization: Converting multiplication into addition makes differentiation straightforward and computationally efficient for optimization algorithms.



Conclusion: MLE requires a product formulation to honor event independence. Transforming product likelihood to log-likelihood ensures stable, convex, and computationally feasible optimization.

Question 2: Error Diagnosis in Perceptron Learning

Scenario: While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

- List possible implementation errors in the learning algorithm.
- Analyze how incorrect learning rate or weight updates affect convergence.
- Identify dataset-related issues that may cause this problem.
- Propose modifications to ensure convergence.

Detailed Engineering Solution & Analysis

Introduction: The Perceptron Learning Algorithm is guaranteed to converge in a finite number of steps (Perceptron Convergence Theorem) if the dataset is strictly linearly separable and the update rules are properly implemented.

1. Possible Implementation Errors:

- Missing or un-updated Bias term, forcing the hyper-plane decision boundary through the origin (0,0).

- Incorrect update logic (e.g., updating weights when a prediction is correct instead of only on misclassification).
- Sign inversion error in the weight update rule: using $w = w - \eta * (y - \hat{y}) * x$ instead of addition $w = w + \eta * (y - \hat{y}) * x$.

2. Impact of Learning Rate & Weight Updates: If learning rate η is extremely high without feature scaling, weight updates overshoot optimal decision boundaries continuously, oscillating back and forth. If weight updates are applied incorrectly or missed due to threshold indexing errors, the weight vector cannot rotate into the target linear separation zone.

3. Dataset & Data Preprocessing Issues:

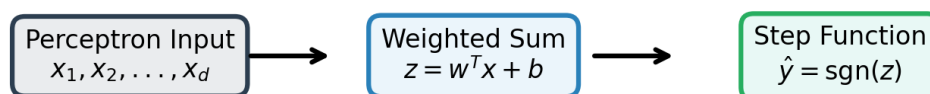
- Non-linearly separable data (e.g., XOR pattern disguised within the data).
- Unscaled or vastly mismatched feature ranges (e.g., Feature 1 $\in [0,1]$, Feature 2 $\in [10^6, 10^7]$), causing severe gradient imbalance.
- Mislabeled ground-truth labels in the dataset introducing conflicting linear constraints.

4. Proposed Modifications for Guaranteed Convergence:

1. Ensure proper bias weight initialization (w_0) and append a constant feature $x_0 = 1$.

2. Apply Feature Normalization (StandardScaler / MinMax) before training.

3. Implement a learning rate decay schedule or switch to Pocket Algorithm / Soft-margin Linear SVM if noisy labels exist.



Convergence Failures: 1. High η (Oscillation) | 2. Non-Separable (XOR) | 3. Missing Bias

Conclusion: Perceptron convergence requires correct update logic, proper bias inclusion, and clean linearly separable data with scaled features.

Question 3: Backpropagation Gradient Error Analysis

Scenario: A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

- Identify possible mathematical or coding errors in gradient computation.
- Explain the role of activation functions in gradient flow.
- Analyze how vanishing or exploding gradients affect training.
- Suggest techniques to fix the issue (e.g., normalization, initialization).

Detailed Engineering Solution & Analysis

Introduction: Backpropagation computes the gradient of the loss function with respect to every parameter using the chain rule of calculus. Divergent or increasing loss indicates numerical instability or gradient corruption.

1. Mathematical and Coding Gradient Errors:

- **Gradient Sign Reversal:** Adding gradients ($w = w + \eta * dW$) instead of subtracting during gradient descent ($w = w - \eta * dW$).

- Incorrect Chain Rule implementation in custom autograd layers (e.g., omitting inner derivatives).
- Inappropriate loss function matching (e.g., using MSE with binary classification sigmoid without soft bounds).

2. Role of Activation Functions in Gradient Flow: Activation functions determine how gradients propagate backward through hidden layers. Saturating activations like Sigmoid $\sigma(z)$ or Tanh have maximum derivative values of 0.25 and 1.0 respectively. When inputs z are large, derivatives approach zero, blocking gradient flow back to early layers.

3. Vanishing and Exploding Gradients:

- **Exploding Gradients:** Unbounded activations (or large initial weight matrices) lead to derivative products > 1 across deep layers. Gradients grow exponentially, causing parameter overflow (NaN/Inf) and rapidly increasing loss.
- **Vanishing Gradients:** Products of small derivatives (< 1) approach zero, freezing parameter learning in initial layers.

4. Corrective Engineering Techniques:

1. **Weight Initialization:** Use Xavier/Glorot initialization for Sigmoid/Tanh, and He Initialization for ReLU activations.

2. **Normalization Layers:** Add Batch Normalization (BatchNorm) or Layer Normalization to maintain activation variance.

3. **Gradient Clipping:** Enforce maximum norm thresholds on gradients (e.g., `clip_grad_norm`) to eliminate gradient explosion.



Backprop Gradient Chain Rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \sigma'(z) \cdot a^{(l-1)}$$

Conclusion: Proper activation selection, controlled parameter initialization, and gradient clipping guarantee smooth backpropagation gradient flow.

Question 4: Gradient Descent vs SGD Error Investigation

Scenario: A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

- Explain why batch gradient descent is slow in large datasets.
- Analyze the cause of fluctuations in SGD.
- Compare the error behavior of Mini-Batch Gradient Descent.
- Recommend the best approach and justify with error trade-offs.

Detailed Engineering Solution & Analysis

Introduction: Optimization algorithms dictate how model parameters are updated based on calculated gradients. Choosing between full-batch, single-sample, and mini-batch strategies involves a trade-off between computational efficiency, convergence speed, and gradient variance.

1. Why Batch Gradient Descent is Slow on Large Datasets: Batch Gradient Descent (BGD) computes gradients across the entire dataset (N samples) to perform a single weight update: $\theta = \theta - \eta * (1/N) \sum \nabla L_i(\theta)$. When N is large (e.g., millions of rows), each step requires massive memory bandwidth and matrix operations, rendering iteration speed extremely slow.

2. Cause of Loss Fluctuations in SGD: Stochastic Gradient Descent (SGD) updates parameters using a single randomly selected sample i per step: $\theta = \theta - \eta * \nabla L_i(\theta)$. Because individual data points contain sample-specific noise and variance, the gradient estimation fluctuates wildly from step to step, producing a noisy learning curve.

3. Error Behavior of Mini-Batch Gradient Descent: Mini-Batch GD splits data into small batches (e.g., $B = 32, 64, 128$). It reduces gradient variance significantly compared to SGD while preserving GPU vectorization efficiency. The loss curve exhibits minor controlled oscillations that help escape shallow local minima without chaotic divergence.

4. Recommended Approach & Justification: Recommendation: Use Mini-Batch Gradient Descent paired with an adaptive optimizer (e.g., Adam or SGD with Momentum).

Justification: Mini-Batch strikes the optimal balance between vectorization memory efficiency, smooth optimization convergence, and escape from saddle points.

Batch GD

- Full Dataset
- Stable & Slow
- High Memory

Stochastic GD (SGD)

- 1 Sample per Step
- Fast, High Variance
- Escapes Local Minima

Mini-Batch GD

- Batch Size (32-256)
- Optimal Convergence
- Vectorized Vectoring

Conclusion: Mini-Batch GD combines the stability of Batch GD with the computational speed and local-minima escape properties of SGD.

Question 5: Curse of Dimensionality Error Analysis

Scenario: A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

- Explain how high dimensionality increases model error.
- Identify signs of overfitting in this scenario.
- Analyze why distance-based learning fails in high dimensions.
- Suggest dimensionality reduction or regularization techniques to improve performance.

Detailed Engineering Solution & Analysis

Introduction: The 'Curse of Dimensionality' refers to geometric and statistical phenomena that arise when analyzing data in high-dimensional spaces. As feature dimensions increase, data becomes increasingly sparse.

1. How High Dimensionality Increases Model Error: As feature dimension d grows, the volume of feature space increases exponentially ($V \propto s^d$). Data points become extremely sparse. Models can easily construct hyper-planes or decision boundaries that memorize random noise patterns in small training samples, driving training error to near zero while causing massive generalization error on test data.

2. Observable Signs of Overfitting:

- Large gap between training performance (e.g., 99% accuracy) and testing performance (e.g., 58% accuracy).

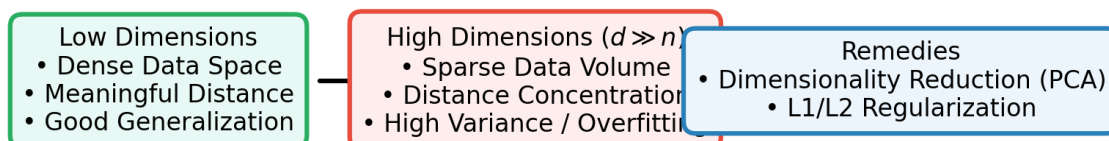
- Extremely large weight coefficients (w_i) in linear/logistic models.
- Model sensitivity to minor noise perturbation in test inputs.

3. Failure of Distance-Based Learning in High Dimensions: In high-dimensional space, the ratio between the distance to the nearest point and the distance to the farthest point approaches 1: $\lim_{d \rightarrow \infty} (D_{\max} - D_{\min}) / D_{\min} = 0$. All data points become virtually equidistant from one another, rendering distance metrics (Euclidean, Manhattan) meaningless for classifiers like k-NN or RBF SVMs.

4. Effective Mitigation Techniques: 1. Feature Selection / Dimensionality Reduction: Apply Principal Component Analysis (PCA) or t-SNE / UMAP to project high-dimensional features into a lower-dimensional latent space.

2. Regularization Penalties: Incorporate L1 (Lasso) regularization to induce sparsity and eliminate uninformative features, or L2 (Ridge) regularization to penalize large weights.

3. Feature Scaling & Dropout: Standardize features and use Dropout layers in deep neural networks.



Conclusion: Mitigating high-dimensional overfitting requires dimensionality reduction (PCA) and strength-controlled regularization (L1/L2) to preserve meaningful feature variance.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand